

Hardware Implementation & Resource Management

Today Agenda

- Review Polling vs Interrupt
- Review Interrupt Hardware Component
- Interrupt Code Example
- RTOS Resource Management

Required Reading

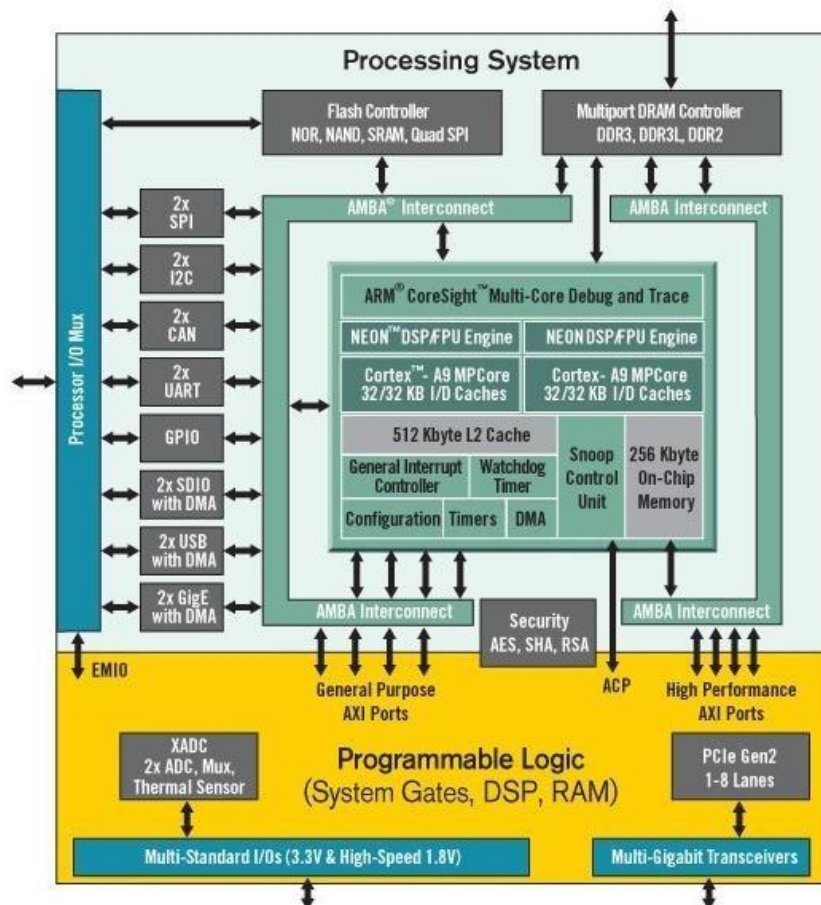
- **The ZYNQ Book**
 - *Chapter 10.4: Interrupts*
- **FreeRTOS Kernel Tutorial:**
 - *Chapter 7*
- **The ZYNQ Book Tutorials**
 - *2A, 2B, 2C*

Interrupt vs Polling

- Basic:
Interrupt: device notify CPU that it needs CPU attention.
Polling: CPU constantly checks devices status whether it needs CPU attention.
- Mechanism:
Interrupt is a hardware mechanism.
Polling is a protocol.
- Efficiency:
Interrupt becomes inefficient when devices keep on interrupt the CPU repeatedly.
Polling becomes inefficient when CPU rarely finds a device ready for service.

Zynq Interrupts – Global Interrupt Controller

- A manual wire connection is made from the PS IRQ to the GPIO INTC.

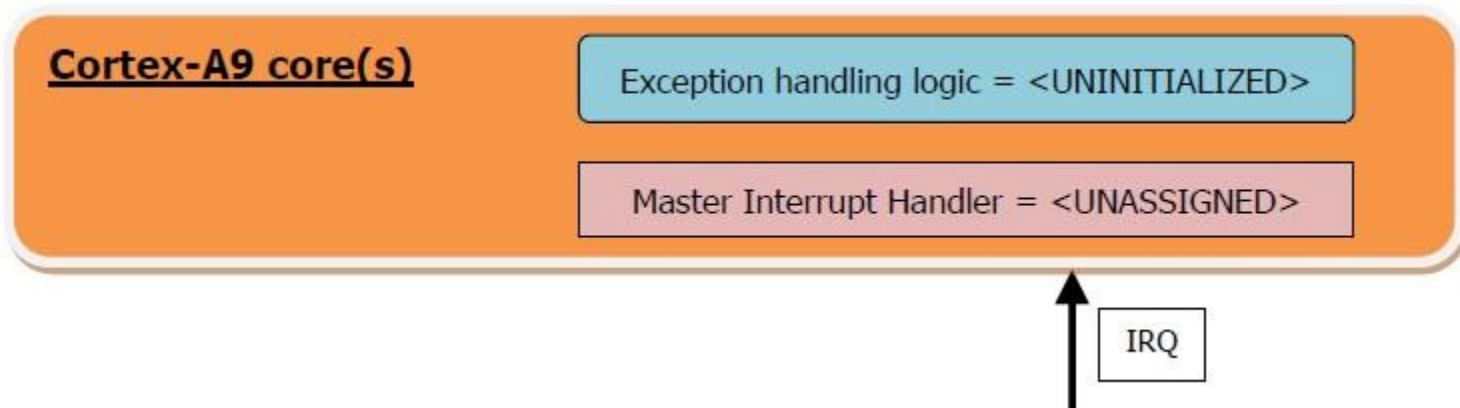


```
Outline Documents Make Target

# XSCUGIC_H
# xstatus.h
# xil_io.h
# xscugic_hw.h
# xil_exception.h
# EFUSE_STATUS_OFFSET
# EFUSE_STATUS_CPU_MASK
# ARMA9
> (anonymous)
T XScuGic_VectorTableEntry : struct
> (anonymous)
T XScuGic_Config : struct
> (anonymous)
T XScuGic : struct
# XScuGic_CPUWriteReg()
# XScuGic_CPURdReg()
# XScuGic_DistWriteReg()
# XScuGic_DistReadReg()
+ XScuGic_Connect(XScuGic*, u32, Xil_InterruptHandler, void*)
+ XScuGic_Disconnect(XScuGic*, u32) : void
+ XScuGic_Enable(XScuGic*, u32) : void
+ XScuGic_Disable(XScuGic*, u32) : void
+ XScuGic_CfgInitialize(XScuGic*, XScuGic_Config*, u32) : void
+ XScuGic_SoftwareIntr(XScuGic*, u32, u32) : s32
+ XScuGic_GetPriorityTriggerType(XScuGic*, u32, u8*, u8*) : void
+ XScuGic_SetPriorityTriggerType(XScuGic*, u32, u8, u8) : void
+ XScuGic_InterruptMapToCpu(XScuGic*, u8, u32) : void
+ XScuGic_Stop(XScuGic*) : void
+ XScuGic_SetCpuID(u32) : void
+ XScuGic_GetCpuID(void) : u32
+ XScuGic_LookupConfig(u16) : XScuGic_Config*
+ XScuGic_InterruptHandler(XScuGic*) : void
+ XScuGic_SelfTest(XScuGic*) : s32
```

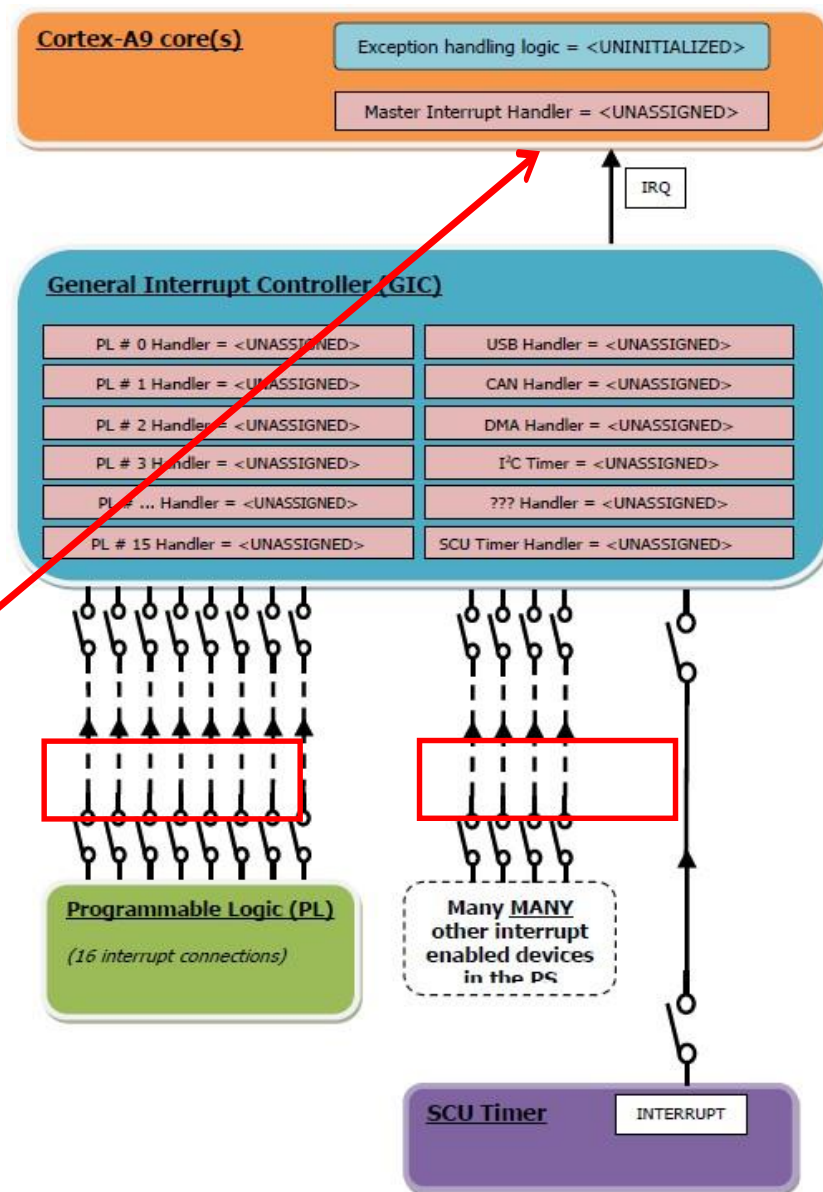
Zynq Interrupts – Global Interrupt Controller

- The Cortex-A9 CPU cores have internal **exception handling logic** which is disabled and initialized at power-up.
- There is one standard interrupt pin on the Cortex-A9 core, and there is a **master interrupt handler** which the CPU executes when receiving any interrupt request (IRQ). The handler is initially unassigned.



Zynq Interrupts – Global Interrupt Controller

- Each peripheral interrupt source must be enabled.
- When an interrupt occurs, the processor has to interrogate the GIC to find out which peripheral generated the interrupt. Done automatically by *XScuGic_InterruptHandler*



Block Design

- Enable GPIO interrupts must be enabled.

Component Name: axi_gpio_0

Board IP Configuration

GPIO

GPIO 2

☐ All Inputs

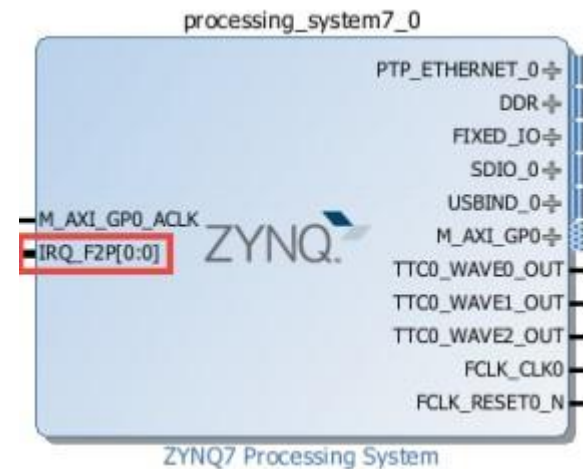
☐ All Outputs

GPIO Width: 32 [1 - 32]

Default Output Value: 0x00000000 [0x00000000, 0xFFFFFFFF]

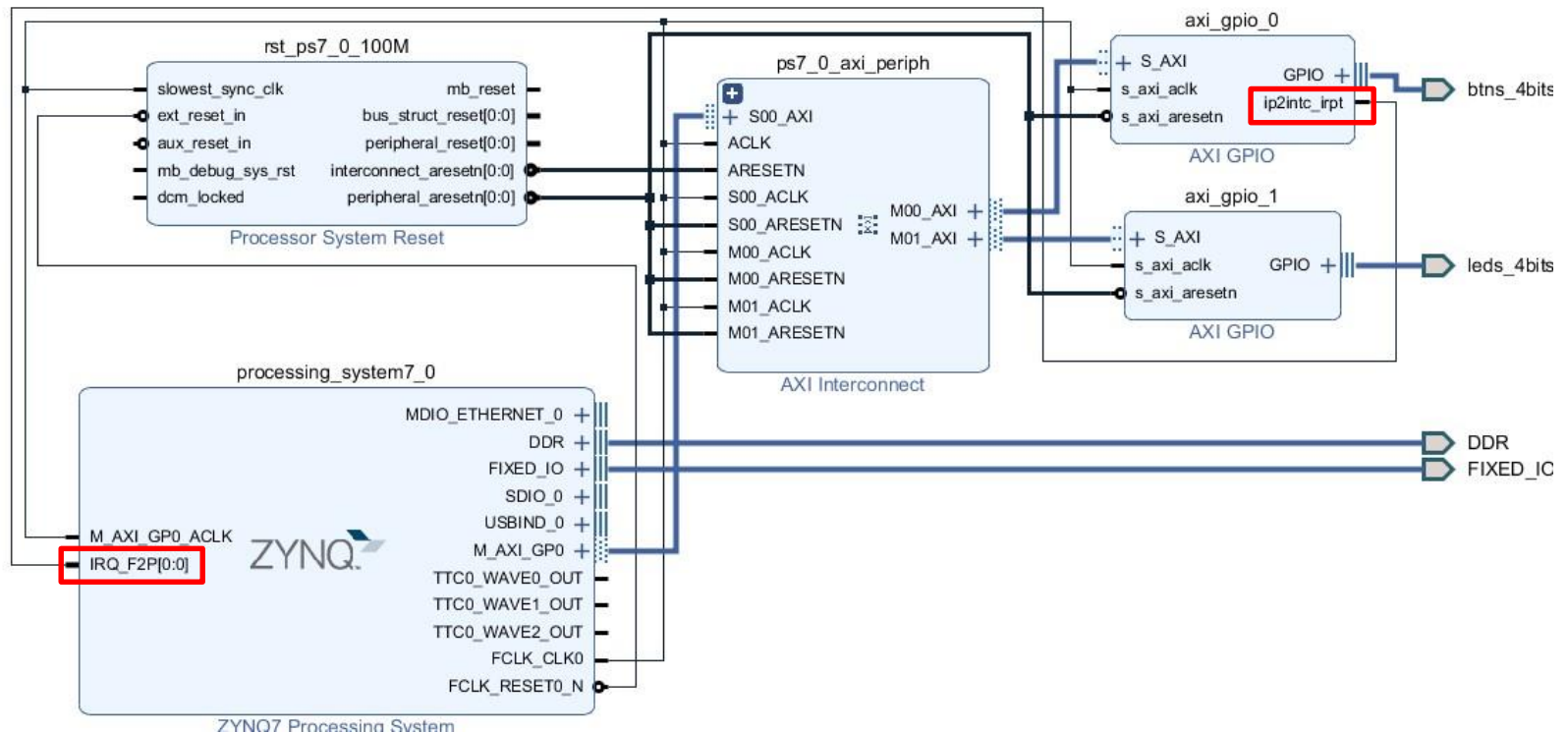
Default Tri State Value: 0xFFFFFFFF [0x00000000, 0xFFFFFFFF]

☒ Enable Interrupt



Block Design

- A manual wire connection is made from the PS IRQ to the GPIO INTC.



C Program (1)

```
#include "xparameters.h"  
#include "xgpio.h"  
#include "xscugic.h"  
#include "xil_exception.h"  
#include "xil_printf.h"
```

```
// Parameter definitions
```

```
#define INTC_DEVICE_ID      XPAR_PS7_SCUGIC_0_DEVICE_ID  
#define BTNS_DEVICE_ID     XPAR_AXI_GPIO_0_DEVICE_ID  
#define LEDS_DEVICE_ID     XPAR_AXI_GPIO_1_DEVICE_ID  
#define INTC_GPIO_INTERRUPT_ID  
    XPAR_FABRIC_AXI_GPIO_0_IP2INTC_IRPT_INTR
```

```
#define BTN_INT                XGPIO_IR_CH1_MASK
```

C Program (2)

```
XGpio LEDInst, BTNInst;
```

```
XScuGic INTCInst;
```

```
static int led_data;
```

```
static int btn_value;
```

```
//-----
```

```
// PROTOTYPE FUNCTIONS
```

```
//-----
```

```
static void BTN_Intr_Handler(void *InstancePtr);
```

```
static int InterruptSystemSetup(XScuGic *XScuGicInstancePtr);
```

```
static int IntcInitFunction(u16 DeviceId, XGpio *GpioInstancePtr);
```

C Program (3)

```
void BTN_Intr_Handler(void *InstancePtr)
{
    // Disable GPIO interrupts
    XGpio_InterruptDisable(&BTNInst, BTN_INT);
    // Ignore additional button presses
    if ((XGpio_InterruptGetStatus(&BTNInst) & BTN_INT) != BTN_INT) {
        return;
    }
    btn_value = XGpio_DiscreteRead(&BTNInst, 1);
    // Increment counter based on button value
    // Reset if center button pressed
    if(btn_value != 8)
        led_data = led_data + btn_value;
    else
        led_data = 0;
    XGpio_DiscreteWrite(&LEDInst, 1, led_data);
    (void) XGpio_InterruptClear(&BTNInst, BTN_INT);
    // Enable GPIO interrupts
    XGpio_InterruptEnable(&BTNInst, BTN_INT);
}
```

C Program (4)

```
int main (void)
{
    int status;
    // Initialize LEDs
    status = XGpio_Initialize(&LEDInst, LEDS_DEVICE_ID);
    if(status != XST_SUCCESS) return XST_FAILURE;
    // Initialize Push Buttons
    status = XGpio_Initialize(&BTNInst, BTNS_DEVICE_ID);
    if(status != XST_SUCCESS) return XST_FAILURE;
    // Set LEDs direction to outputs
    XGpio_SetDataDirection(&LEDInst, 1, 0x00);
    // Set all buttons direction to inputs
    XGpio_SetDataDirection(&BTNInst, 1, 0xFF);

    // Initialize interrupt controller
    status = IntcInitFunction(INTC_DEVICE_ID, &BTNInst);
    if(status != XST_SUCCESS) return XST_FAILURE;
    while(1);
    return 0;
}
```

C Program

(5)

```
int IntcInitFunction(u16 DeviceId, XGpio *GpioInstancePtr)
{
    XScuGic_Config *IntcConfig;
    int status;

    // Interrupt controller initialization
    IntcConfig = XScuGic_LookupConfig(DeviceId);
    status = XScuGic_CfgInitialize(&INTCInst, IntcConfig,
                                   IntcConfig->CpuBaseAddress);
    if(status != XST_SUCCESS) return XST_FAILURE;

    // Call to interrupt setup
    status = InterruptSystemSetup(&INTCInst);
    if(status != XST_SUCCESS) return XST_FAILURE;
```

C Program (6)

```
// Connect GPIO interrupt to handler
```

```
status = XScuGic_Connect(&INTCInst, INTC_GPIO_INTERRUPT_ID,  
                        (Xil_ExceptionHandler) BTN_Intr_Handler,  
                        (void *)GpioInstancePtr);
```

```
if(status != XST_SUCCESS) return XST_FAILURE;
```

```
// Enable GPIO interrupts interrupt
```

```
XGpio_InterruptEnable(GpioInstancePtr, 1);
```

```
XGpio_InterruptGlobalEnable(GpioInstancePtr);
```

```
// Enable GPIO interrupts in the controller
```

```
XScuGic_Enable(&INTCInst, INTC_GPIO_INTERRUPT_ID);
```

```
return XST_SUCCESS;
```

```
}
```

C Program (7)

```
int InterruptSystemSetup(XScuGic *XScuGicInstancePtr)
{
    // Enable interrupt
    XGpio_InterruptEnable(&BTNInst, BTN_INT);
    XGpio_InterruptGlobalEnable(&BTNInst);

    Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
                                (Xil_ExceptionHandler) XScuGic_InterruptHandler,
                                XScuGicInstancePtr);

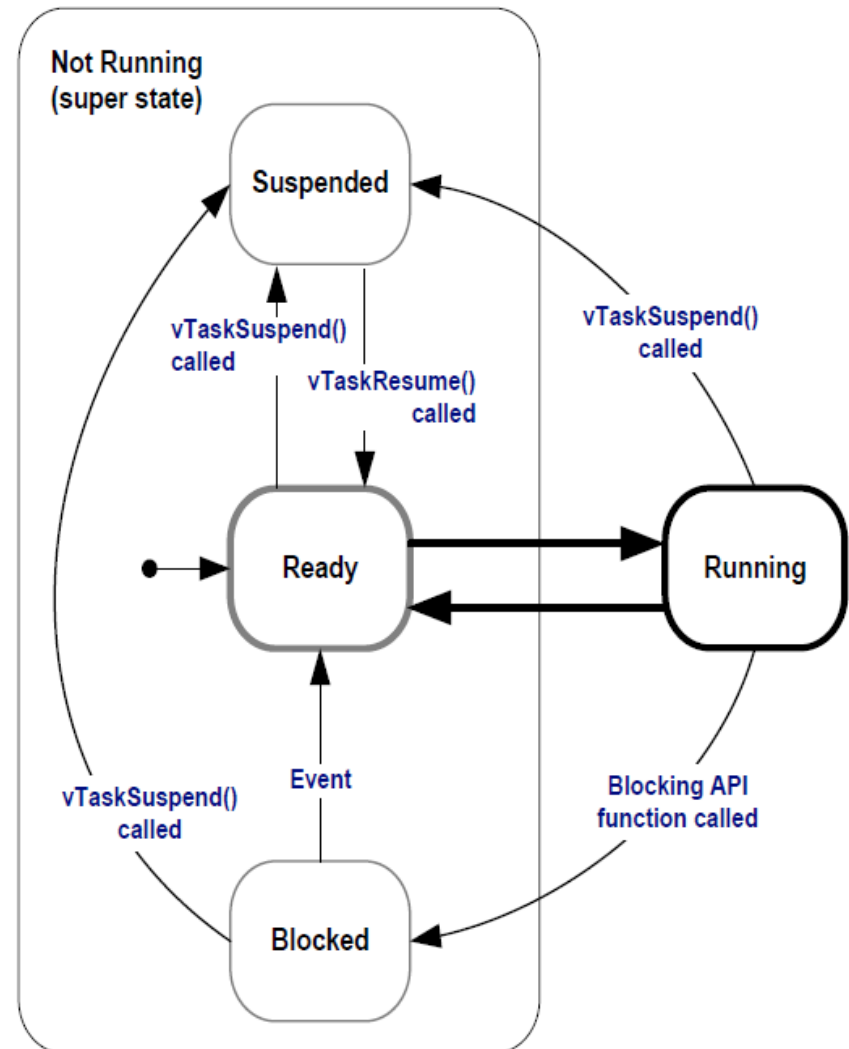
    Xil_ExceptionEnable();

    return XST_SUCCESS;
}
```


Introduction to FreeRTOS

Chapter 7

Resource Management



Resource Management in FreeRTOS

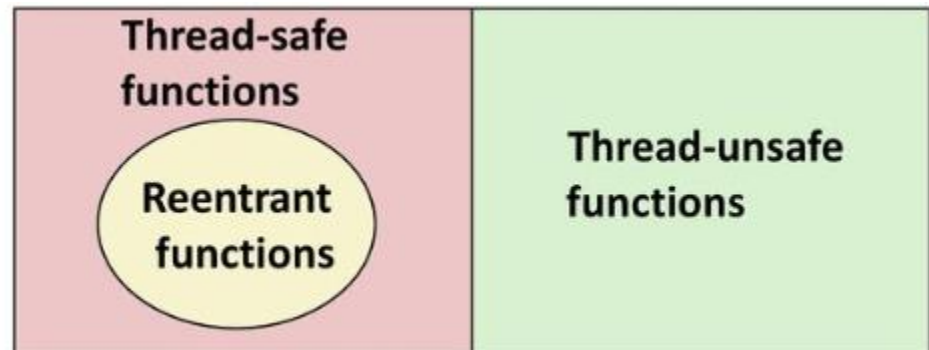
In a multitasking system there is potential for error if one task starts to access a resource like a 4-bit LED, but does not complete its access before being transitioned out of the Running state.

If the task leaves the resource in an inconsistent state, then access to the same resource by any other task could result in data corruption.



Resource Management in FreeRTOS with Function Reentrancy

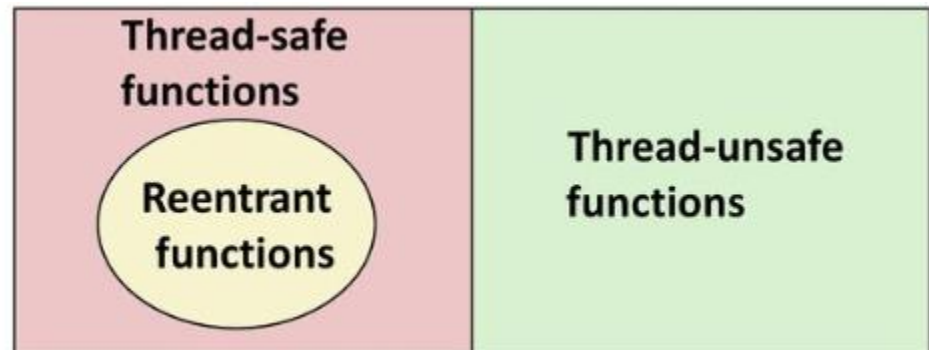
A function is *reentrant* if it is safe to call the function from more than one task. Reentrant functions are said to be *thread safe* because they can be accessed from more than one thread of execution without the risk of data or logical operations becoming corrupted.



Resource Management in FreeRTOS with Function Reentrancy

Each task maintains its own stack and its own set of processor (hardware) register values.

If a function does not access any data other than data stored on the stack or held in a register, then the function is reentrant and thread safe.





Resource Management in FreeRTOS with Function Reentrancy

Example of a reentrant function:

```
/* A parameter is passed into the function. This will either be passed on the stack,
or in a processor register. Either way is safe as each task or interrupt that calls
the function maintains its own stack and its own set of register values, so each task
or interrupt that calls the function will have its own copy of lVar1. */
long lAddOneHundred( long lVar1 )
{
/* This function scope variable will also be allocated to the stack or a register,
depending on the compiler and optimization level. Each task or interrupt that calls
this function will have its own copy of lVar2. */
long lVar2;

    lVar2 = lVar1 + 100;
    return lVar2;
}
```

Listing 112



Resource Management in FreeRTOS with Function Reentrancy

Example of a function that is not reentrant Listing 113

```
/* In this case lVar1 is a global variable, so every task that calls
lNonsenseFunction will access the same single copy of the variable. */
long lVar1;

long lNonsenseFunction( void )
{
/* lState is static, so is not allocated on the stack. Each task that calls this
function will access the same single copy of the variable. */
static long lState = 0;
long lReturn;

    switch( lState )
    {
        case 0 : lReturn = lVar1 + 10;
                  lState = 1;
                  break;

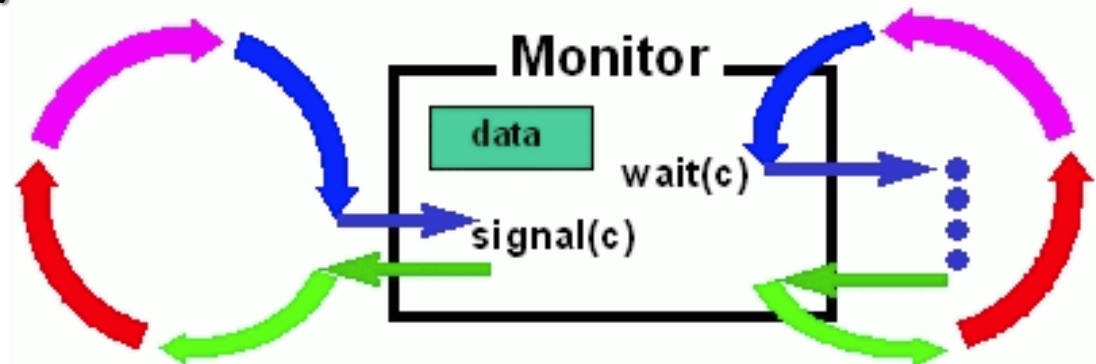
        case 1 : lReturn = lVar1 + 20;
                  lState = 0;
                  break;
    }
}
```




Resource Management in FreeRTOS with Mutual Exclusion

To ensure data consistency is maintained at all times access to a resource that is shared between tasks must be managed using a *mutual exclusion* technique.

The goal is to ensure that, once a task starts to access a shared resource that is not re-entrant and not thread-safe, the same task has exclusive access to the resource until the resource has been returned to a consistent state.

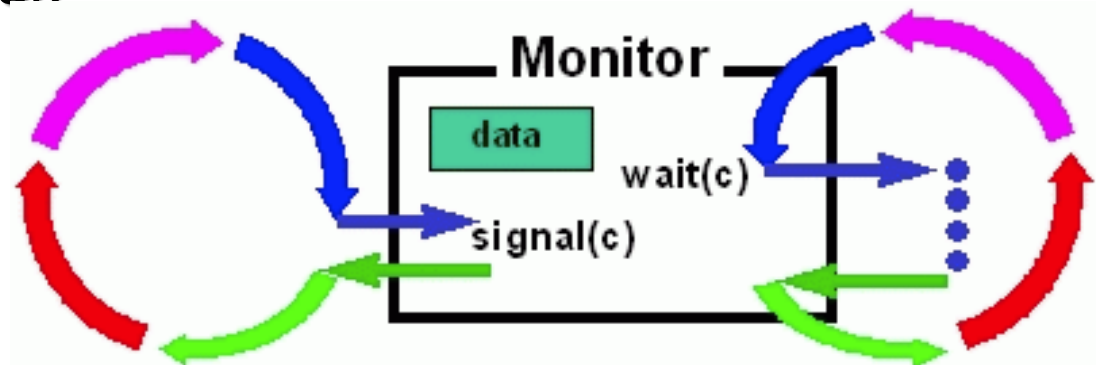


Resource Management in FreeRTOS with Mutual Exclusion



FreeRTOS provides several features that can be used to implement mutual exclusion.

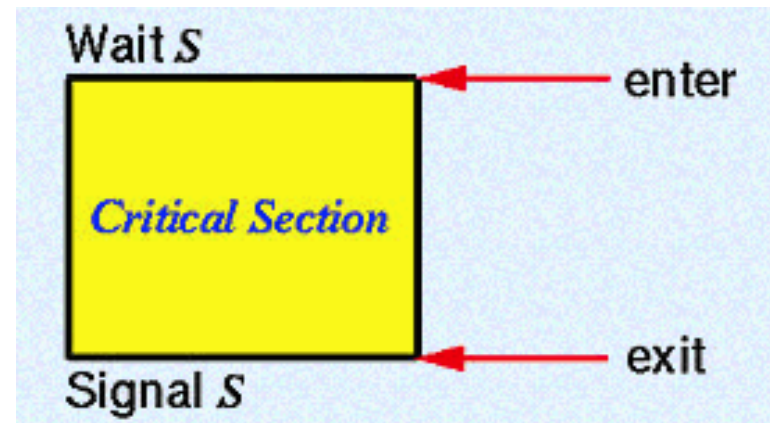
But the best mutual exclusion method is to, whenever possible, design the application in such a way that resources are not shared, or each resource is accessed only from a single task at a time.



Resource Management in FreeRTOS with Critical Sections

Basic *critical sections* are regions of code that are surrounded by calls to the macros *taskENTER_CRITICAL()* and *taskEXIT_CRITICAL()*, respectively.

Critical sections are also known as *critical regions*.





Resource Management in FreeRTOS with Critical Sections

`taskENTER_CRITICAL()` and `taskEXIT_CRITICAL()` do not take any parameters or return a value.

Listing 114

```
/* Ensure access to the PORTA register cannot be interrupted by placing it within a
critical section. Enter the critical section. */
taskENTER_CRITICAL();

/* A switch to another task cannot occur between the call to taskENTER_CRITICAL() and
the call to taskEXIT_CRITICAL(). Interrupts may still execute on FreeRTOS ports that
allow interrupt nesting, but only interrupts whose logical priority is above the
value assigned to the configMAX_SYSCALL_INTERRUPT_PRIORITY constant – and those
interrupts are not permitted to call FreeRTOS API functions. */
PORTA |= 0x01;

/* Access to PORTA has finished, so it is safe to exit the critical section. */
taskEXIT_CRITICAL();
```



Resource Management in FreeRTOS with Critical Sections

The example projects use *vPrintString()* to write strings to standard out. *vPrintString()* is called from many different tasks so, in theory, its implementation could protect access to standard out using a critical section.

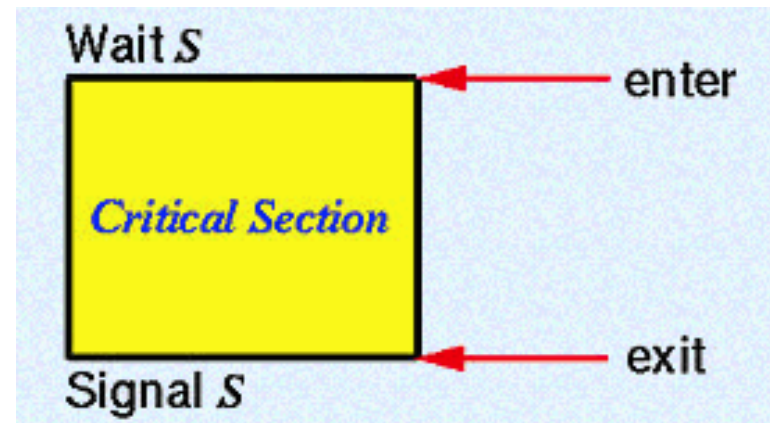
```
void vPrintString( const char *pcString )
{
    /* Write the string to stdout, using a critical section as a crude method of
    mutual exclusion. */
    taskENTER_CRITICAL();
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    taskEXIT_CRITICAL();
}
```

Listing 115

Resource Management in FreeRTOS with Critical Sections

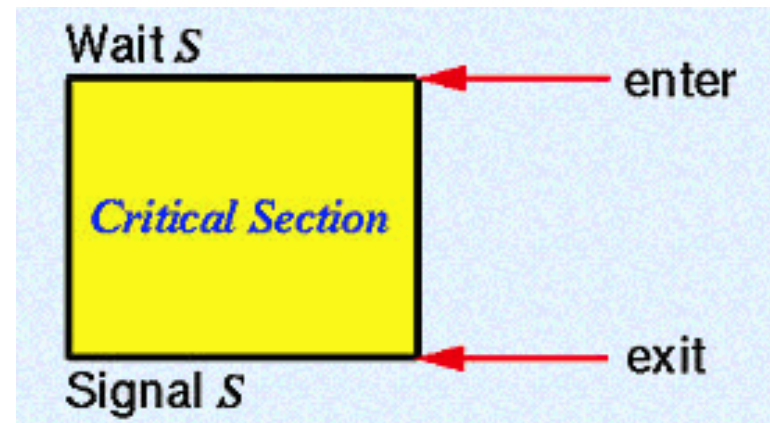
Basic critical sections must be kept very short, otherwise they will adversely affect task management.

Every call to *taskENTER_CRITICAL()* must be closely paired with a call to *taskEXIT_CRITICAL()*.



Resource Management in FreeRTOS with Critical Sections

Critical sections can be *nested*, because the kernel keeps a count of the nesting depth. The critical section will be exited only when the nesting depth returns to zero or when one call to `taskEXIT_CRITICAL()` has been executed for every preceding call to `taskENTER_CRITICAL()`.



Resource Management in FreeRTOS with Mutexes

A *mutex* is a special type of binary semaphore that is used to control access to a resource that is shared between two or more tasks. The word MUTEX originates from *MUTual EXclusion*.

configUSE_MUTEXES must be set to 1 in FreeRTOSConfig.h for mutexes to be available.

```
#define configUSE_MUTEXES  
1
```

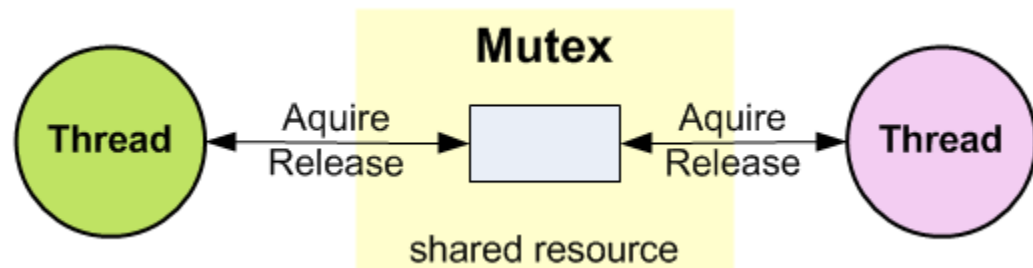




Resource Management in FreeRTOS with Mutexes

When used in a mutual exclusion scenario, the mutex can be thought of as a *token* that is associated with the resource being shared.

For a task to access the resource legitimately, it must first successfully *take the token* or be the *token holder*.

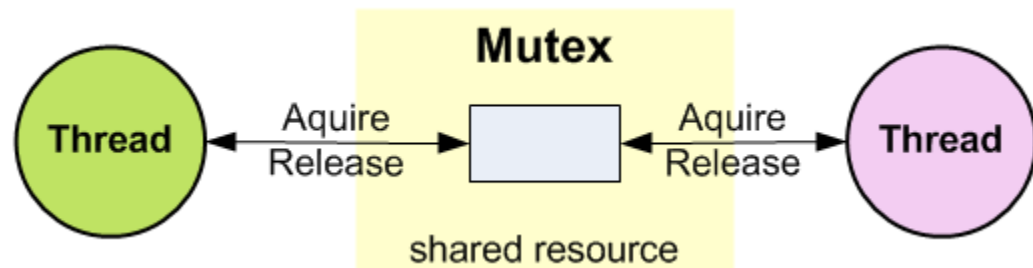




Resource Management in FreeRTOS with Mutexes

When the token holder has finished with the resource, it must *give* the token back. Only when the token has been returned can another task successfully take the token, and then safely access the same shared resource.

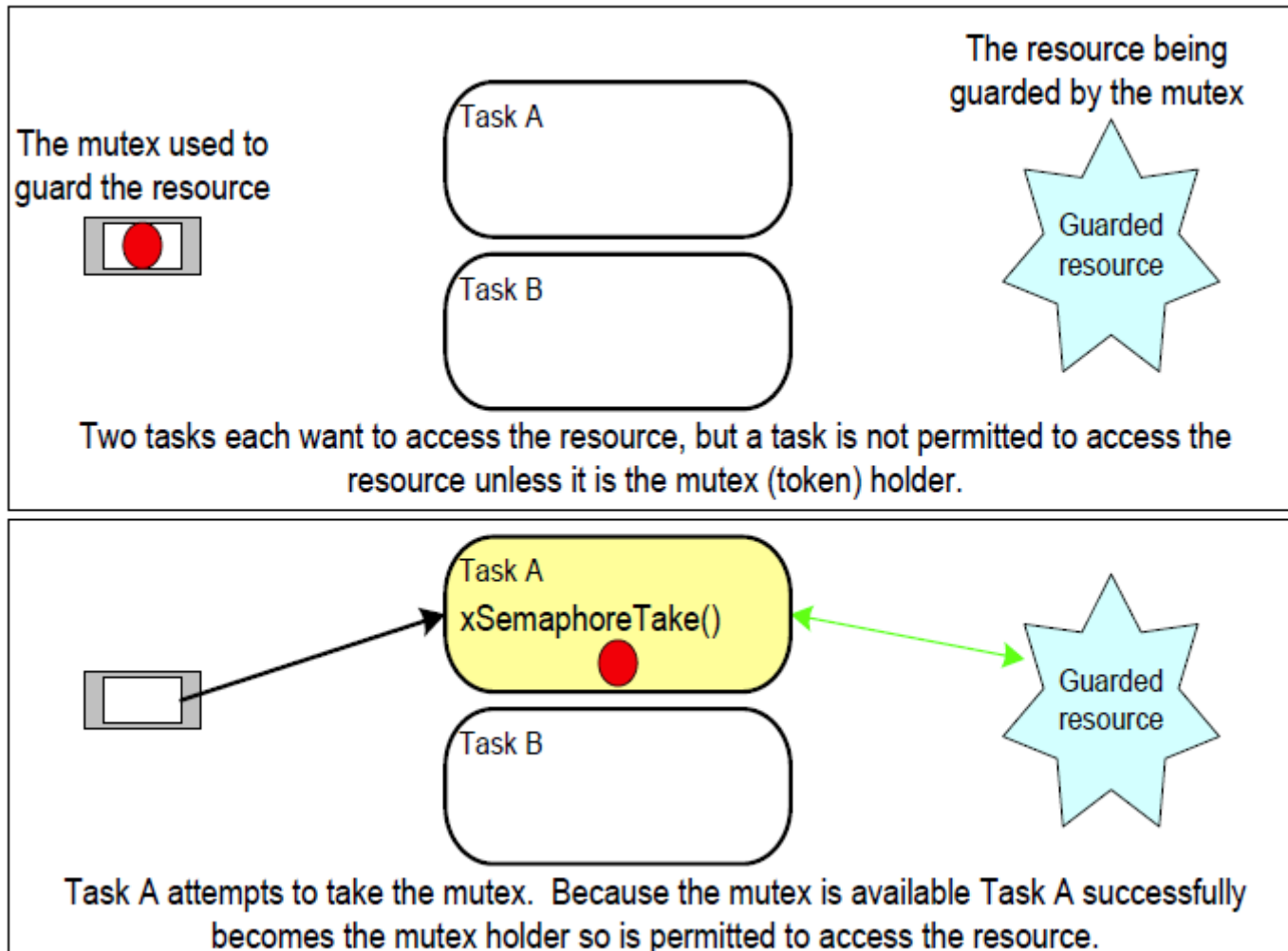
A task is not permitted to access the shared resource unless it holds the token.



Resource Management in FreeRTOS with Mutexes



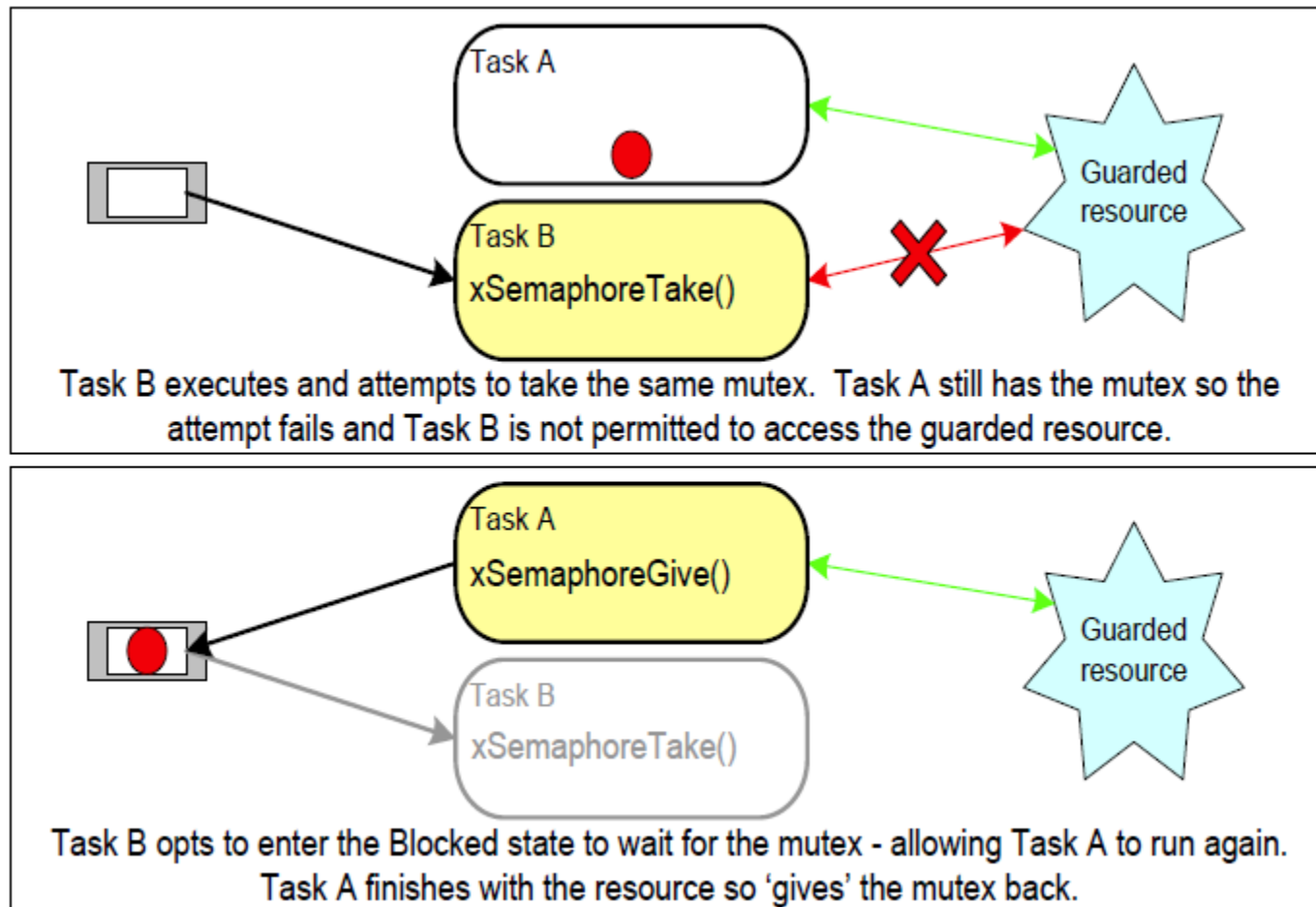
Figure 63



Resource Management in FreeRTOS with Mutexes



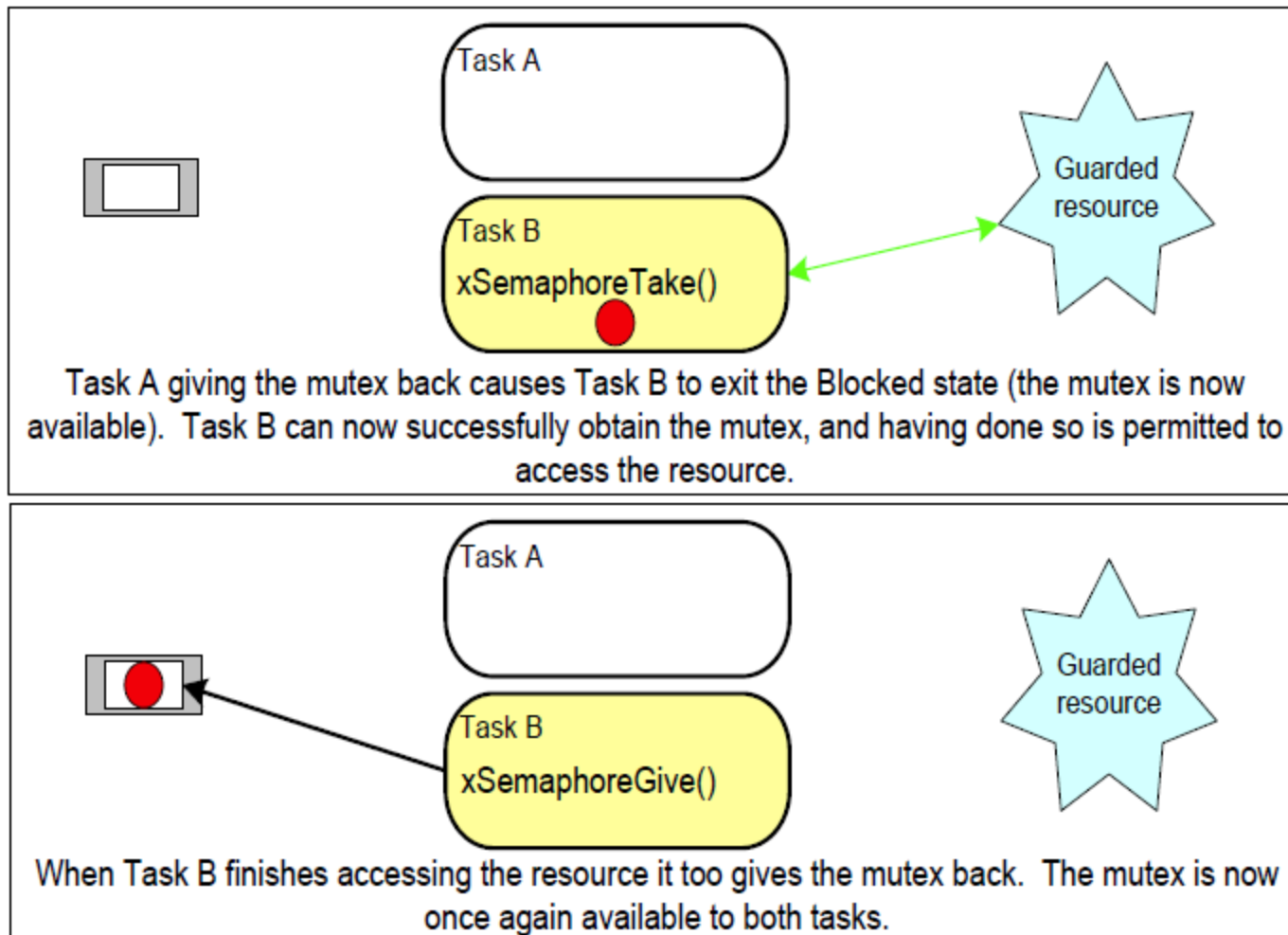
Figure 63



Resource Management in FreeRTOS with Mutexes



Figure 63

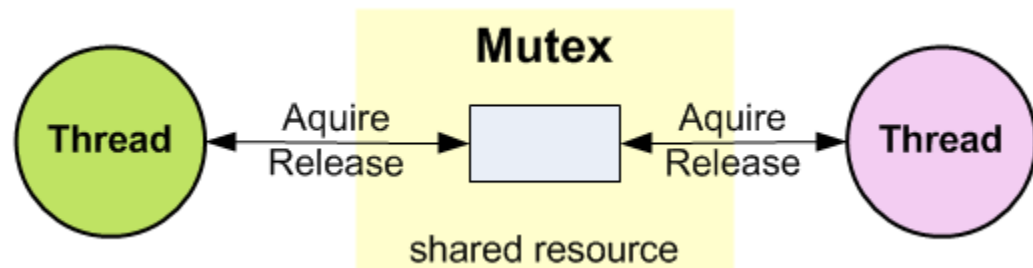




Resource Management in FreeRTOS with Mutexes

A mutex is a type of semaphore. Handles to FreeRTOS semaphore are stored in a variable of type `SemaphoreHandle_t`.

Before a mutex can be used, it must be created. To create a mutex type semaphore, use the `xSemaphoreCreateMutex()` API function.

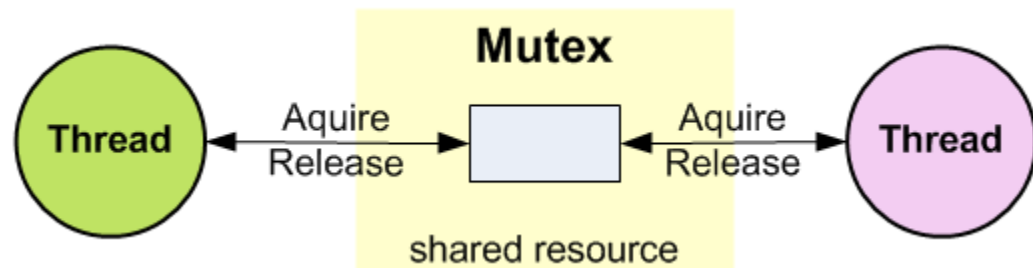




Resource Management in FreeRTOS with Mutexes

The next example creates a version of `vPrintString()` called `prvNewPrintString()`, then calls the new function from multiple tasks.

`prvNewPrintString()` is functionally identical to `vPrintString()` but controls access to standard out using a mutex, rather than by locking the scheduler.



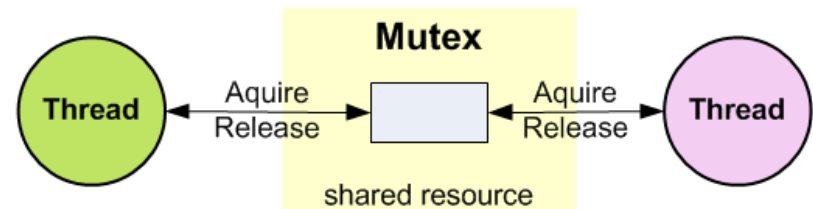


Resource Management in FreeRTOS with Mutexes

```
static void prvNewPrintString( const char *pcString )
{
    /* The mutex is created before the scheduler is started, so already exists by the
    time this task executes.

    Attempt to take the mutex, blocking indefinitely to wait for the mutex if it is
    not available straight away. The call to xSemaphoreTake() will only return when
    the mutex has been successfully obtained, so there is no need to check the
    function return value. If any other delay period was used then the code must
    check that xSemaphoreTake() returns pdTRUE before accessing the shared resource
    (which in this case is standard out). As noted earlier in this book, indefinite
    time outs are not recommended for production code. */
    xSemaphoreTake( xMutex, portMAX_DELAY );
    {
        /* The following line will only execute once the mutex has been successfully
        obtained. Standard out can be accessed freely now as only one task can have
        the mutex at any one time. */
        printf( "%s", pcString );
        fflush( stdout );

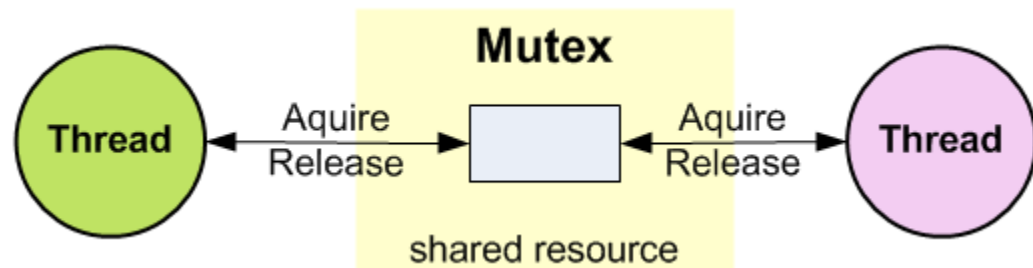
        /* The mutex MUST be given back! */
    }
    xSemaphoreGive( xMutex );
}
```





Resource Management in FreeRTOS with Mutexes

`prvNewPrintString()` is called repeatedly by two instances of a task implemented by `prvPrintTask()`. A random delay time is used between each call. The task parameter is used to pass a unique string into each instance of the task.



Resource Management in FreeRTOS with Mutexes



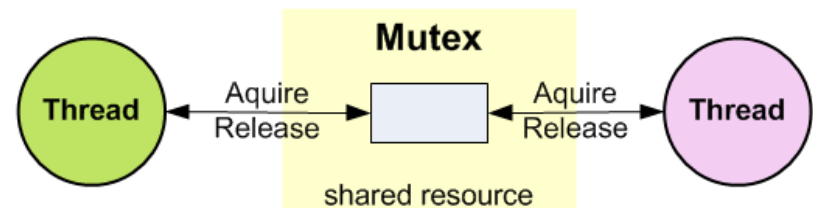
Listing 121

```
static void prvPrintTask( void *pvParameters )
{
    char *pcStringToPrint;
    const TickType_t xMaxBlockTimeTicks = 0x20;

    /* Two instances of this task are created. The string printed by the task is
    passed into the task using the task's parameter. The parameter is cast to the
    required type. */
    pcStringToPrint = ( char * ) pvParameters;

    for( ;; )
    {
        /* Print out the string using the newly defined function. */
        prvNewPrintString( pcStringToPrint );

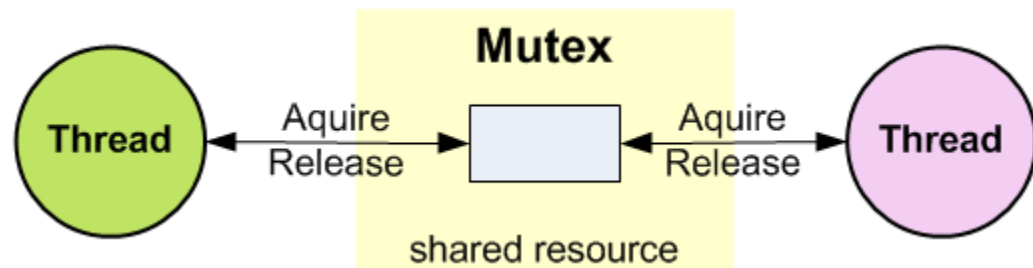
        /* Wait a pseudo random time. Note that rand() is not necessarily reentrant,
        but in this case it does not really matter as the code does not care what
        value is returned. In a more secure application a version of rand() that is
        known to be reentrant should be used - or calls to rand() should be protected
        using a critical section. */
        vTaskDelay( ( rand() % xMaxBlockTimeTicks ) );
    }
}
```



Resource Management in FreeRTOS with Mutexes



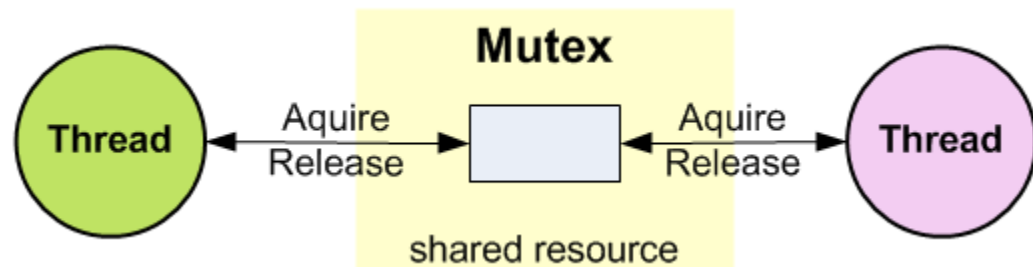
`main()` creates the mutex, creates the tasks, then starts the scheduler. The two instances of `prvPrintTask()` are created at different priorities, so the lower priority task will sometimes be pre-empted by the higher priority task.





Resource Management in FreeRTOS with Mutexes

As a mutex is used to ensure each task gets mutually exclusive access to the terminal, even when pre-emption occurs, the strings that are displayed will be correct and in no way corrupted.





Resource Management in FreeRTOS with Mutexes

Listing 123

```
int main( void )
{
    /* Before a semaphore is used it must be explicitly created. In this example a
    mutex type semaphore is created. */
    xMutex = xSemaphoreCreateMutex();

    /* Check the semaphore was created successfully before creating the tasks. */
    if( xMutex != NULL )
    {
        /* Create two instances of the tasks that write to stdout. The string they
        write is passed in to the task as the task's parameter. The tasks are
        created at different priorities so some pre-emption will occur. */
        xTaskCreate( prvPrintTask, "Print1", 1000,
                    "Task 1 *****\r\n", 1, NULL );

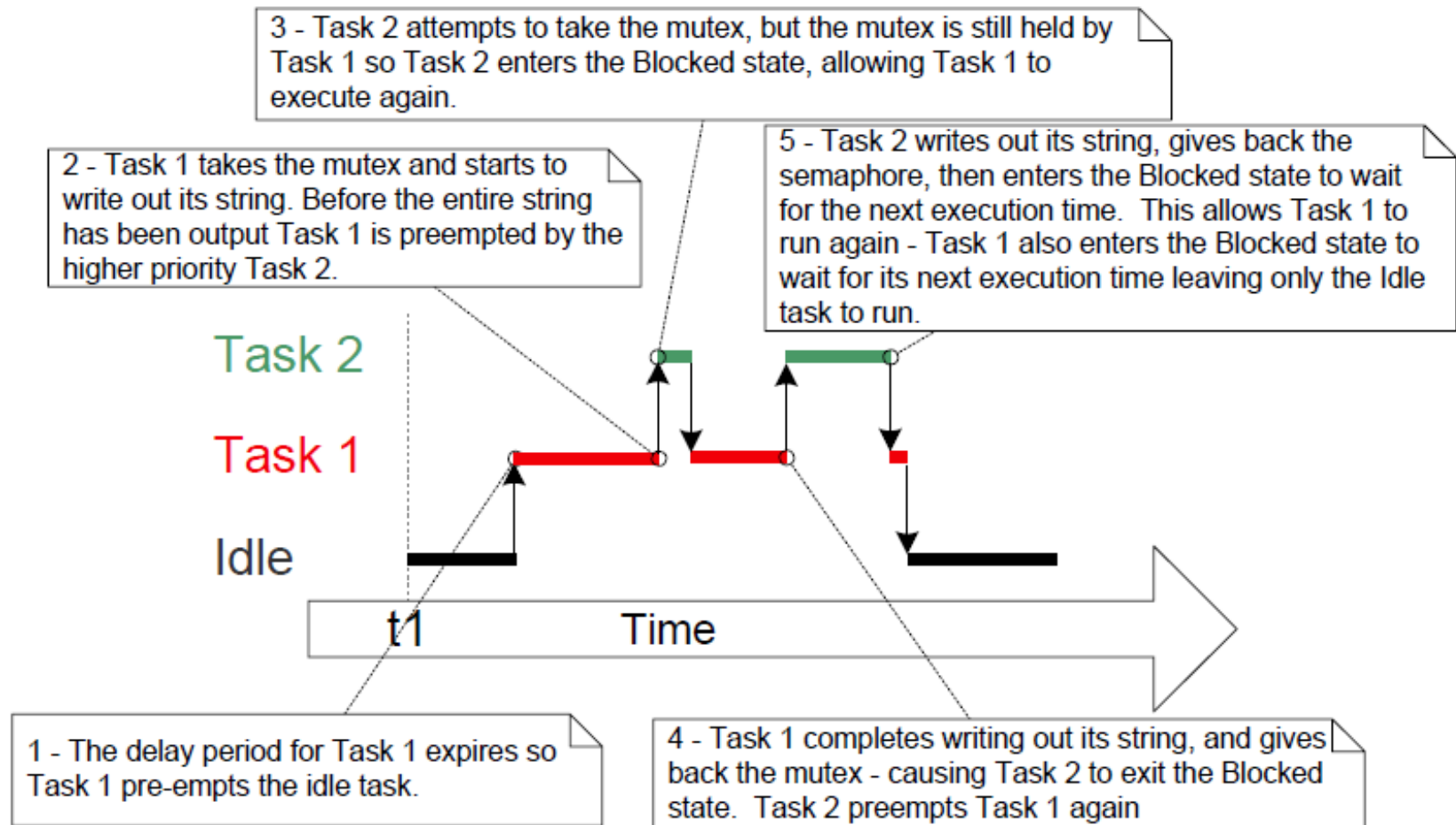
        xTaskCreate( prvPrintTask, "Print2", 1000,
                    "Task 2 ----- \r\n", 2, NULL );

        /* Start the scheduler so the created tasks start executing. */
        vTaskStartScheduler();
    }

    /* If all is well then main() will never reach here as the scheduler will now be
    running the tasks. If main() does reach here then it is likely that there was
    insufficient heap memory available for the idle task to be created. Chapter 2
    provides more information on heap memory management. */
    for( ;; );
}
```



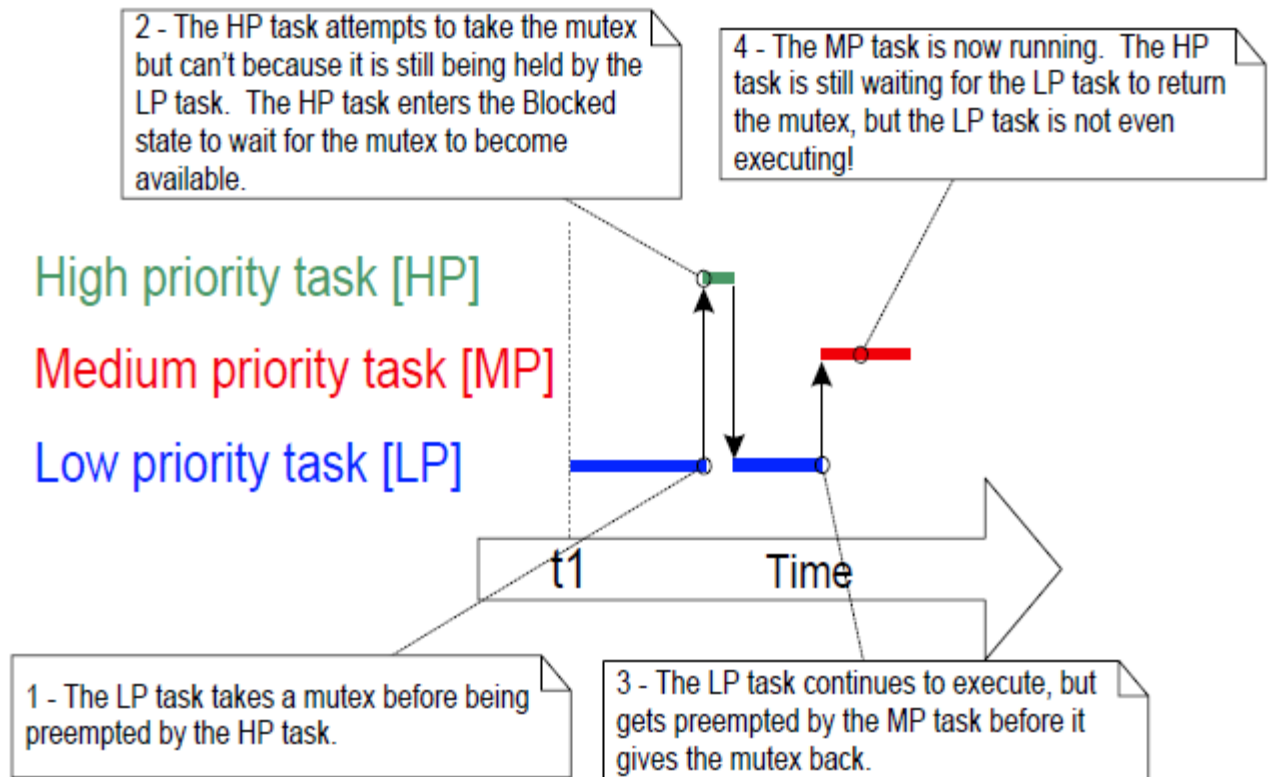
Resource Management in FreeRTOS with Mutexes





Resource Management in FreeRTOS with Mutexes

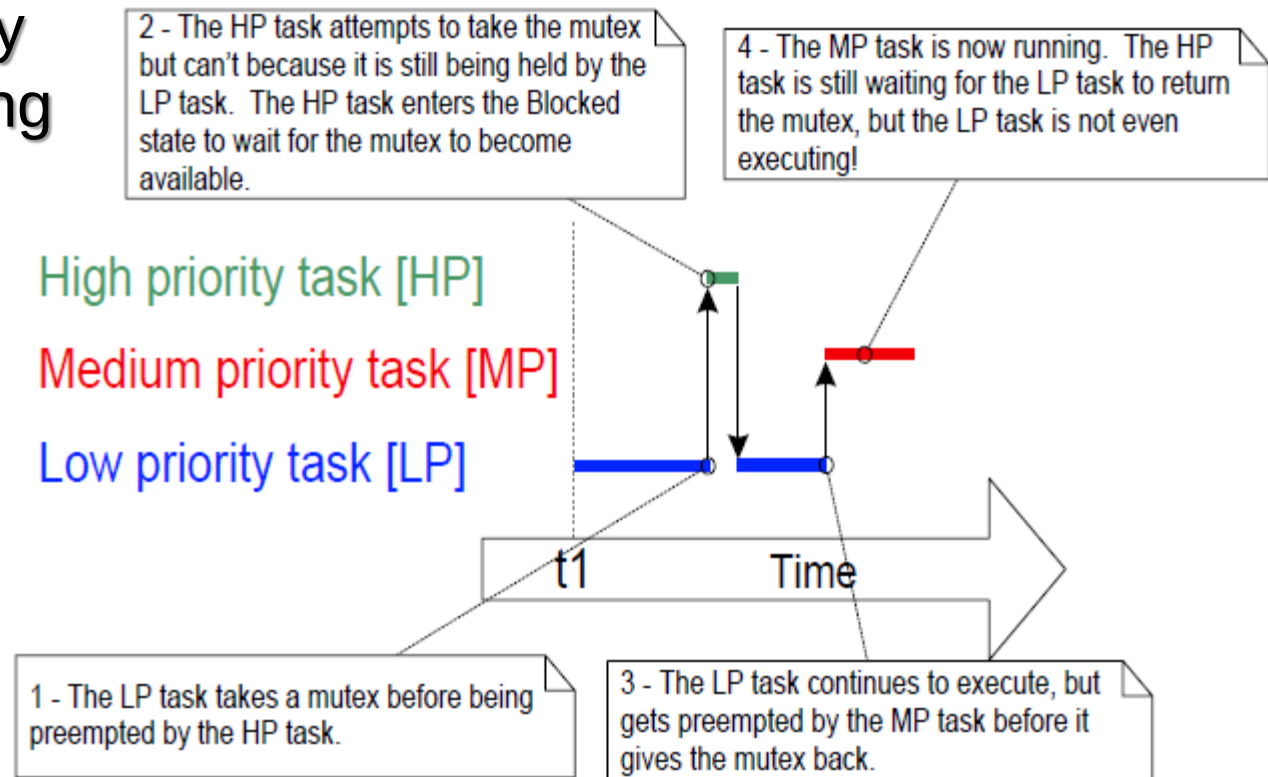
Priority Inversion one of the potential pitfalls of using a mutex to provide mutual exclusion and shows the higher priority Task 2 having to wait for the lower priority Task 1 to give up control of the mutex.





Resource Management in FreeRTOS with Mutexes

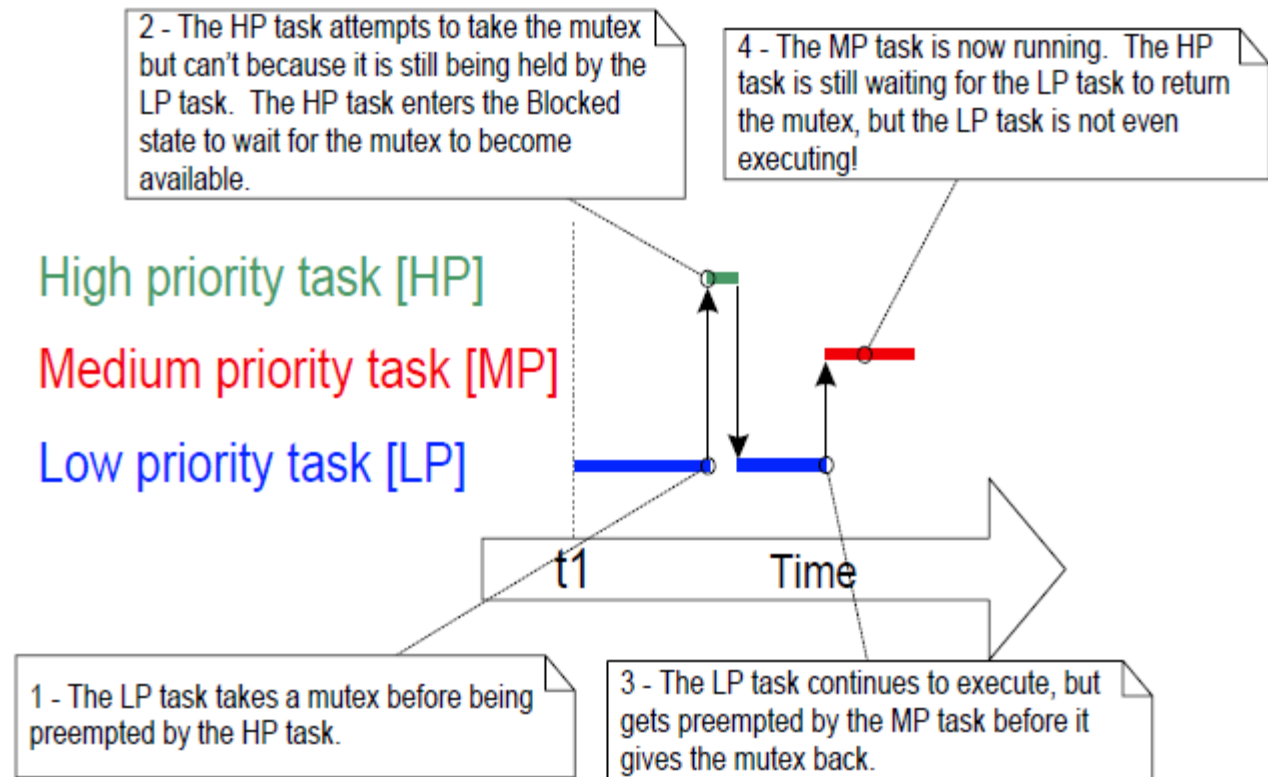
This undesirable priority inversion would be exaggerated further if a medium priority task started to execute while the high priority task was waiting for the semaphore.





Resource Management in FreeRTOS with Mutexes

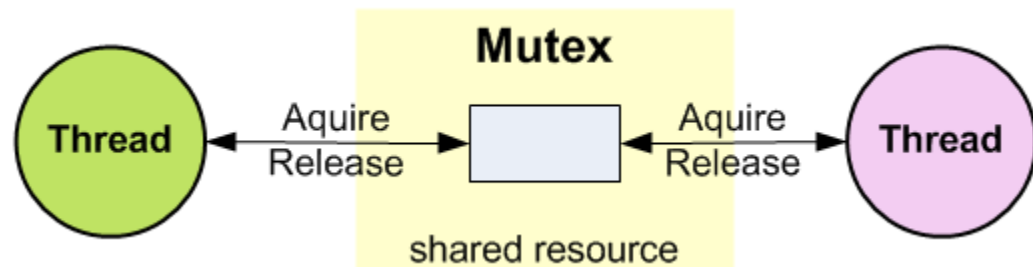
The result would be a high priority task waiting for a low priority task without the low priority task even being able to execute.



Resource Management in FreeRTOS with Mutexes



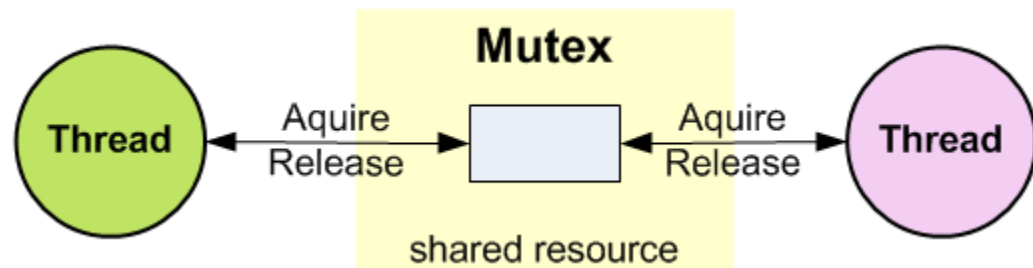
FreeRTOS mutexes and binary semaphores are very similar. The difference is that mutexes include a basic *priority inheritance* mechanism, whereas binary semaphores do not.



Resource Management in FreeRTOS with Mutexes



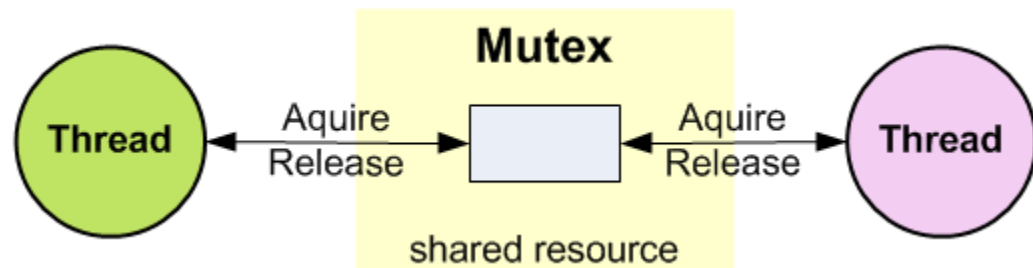
Priority inheritance minimizes the negative effects of priority inversion. It does not 'fix' priority inversion, but merely lessens its impact by ensuring that the inversion is always time bounded.



Resource Management in FreeRTOS with Mutexes



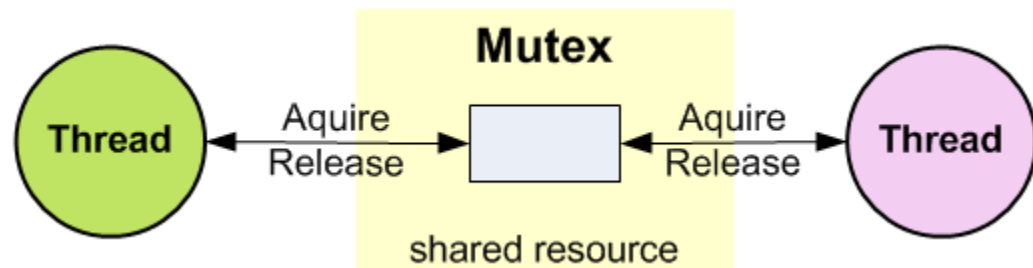
Priority inheritance works by temporarily raising the priority of the mutex holder to the priority of the highest priority task that is attempting to obtain the same mutex.



Resource Management in FreeRTOS with Mutexes

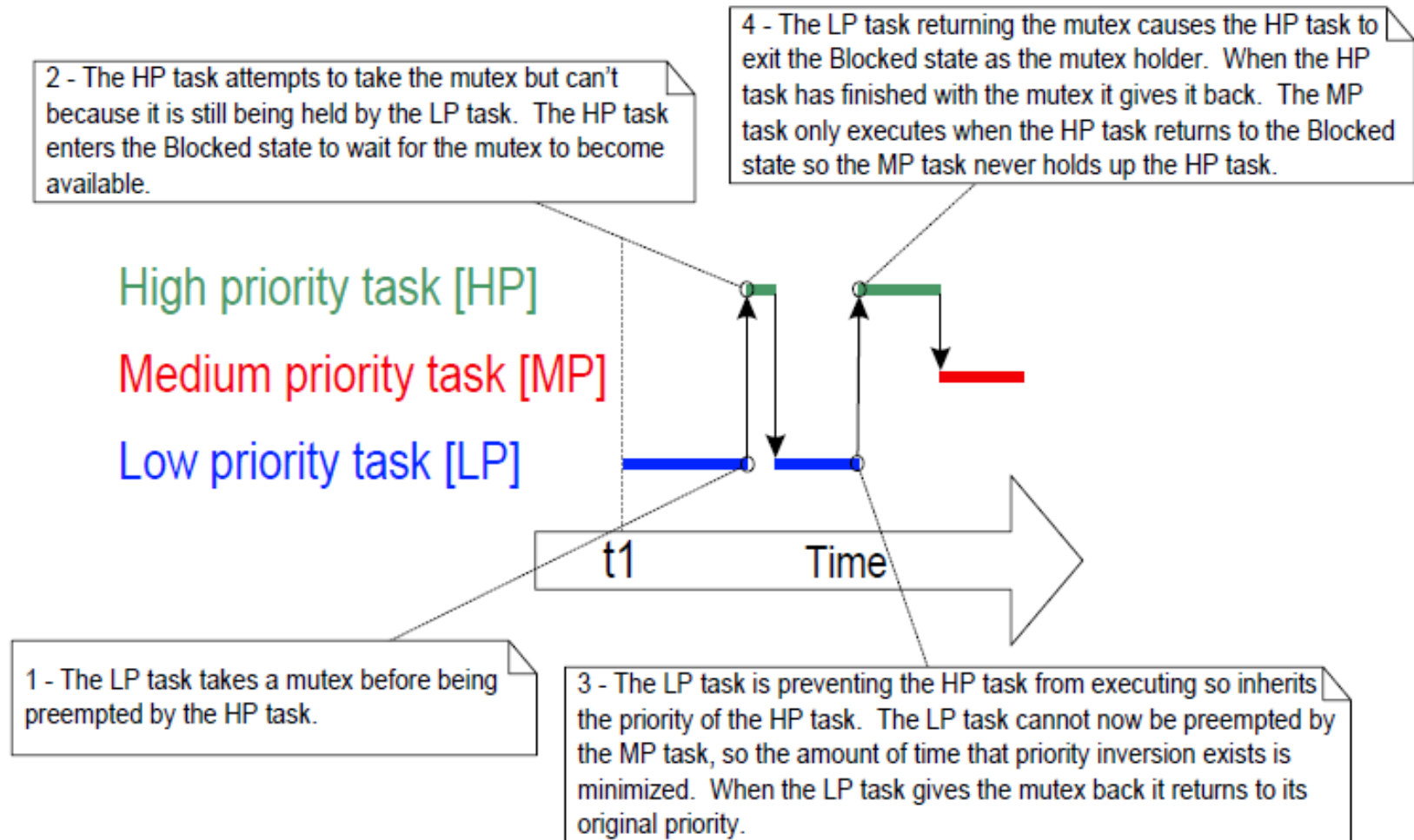


The low priority task that holds the mutex *inherits the priority* of the task waiting for the mutex. The priority of the mutex holder is reset automatically to its original value when it gives the mutex back.





Resource Management in FreeRTOS with Mutexes

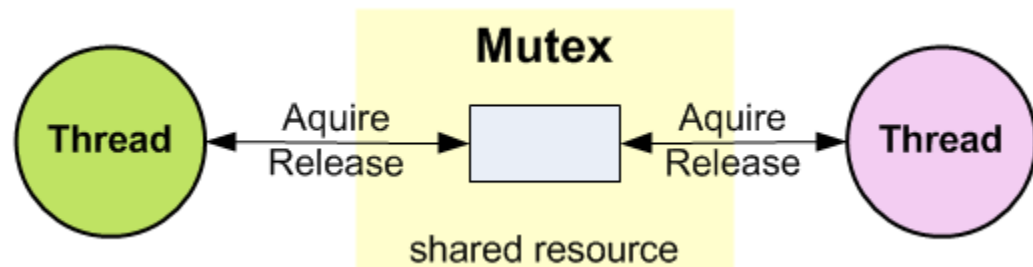


Resource Management in FreeRTOS with Mutexes



Deadlock occurs when two tasks cannot proceed because they are both waiting for a resource that is held by the other.

For example, assume that Task A and Task B both need to acquire mutex X *and* mutex Y in order to perform an action.

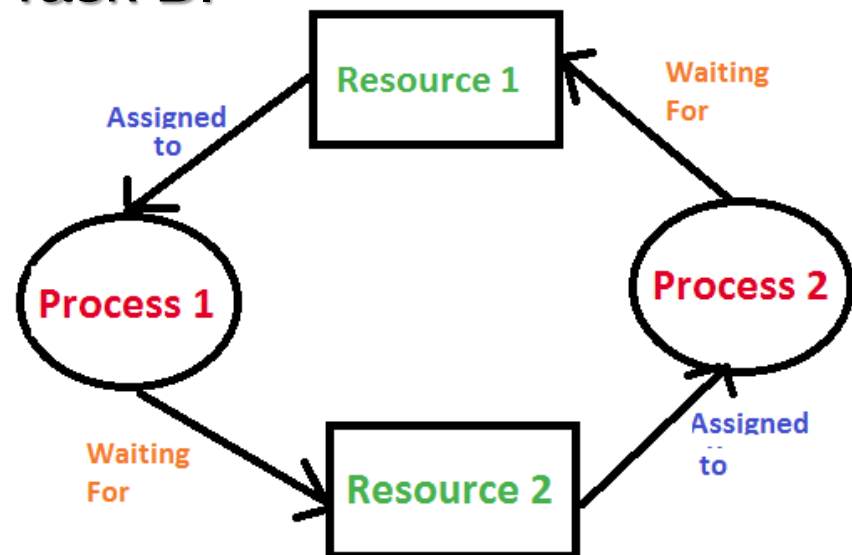




Resource Management in FreeRTOS with Mutexes

Deadlock:

1. Task A executes and successfully takes mutex X.
2. Task A is pre-empted by Task B.



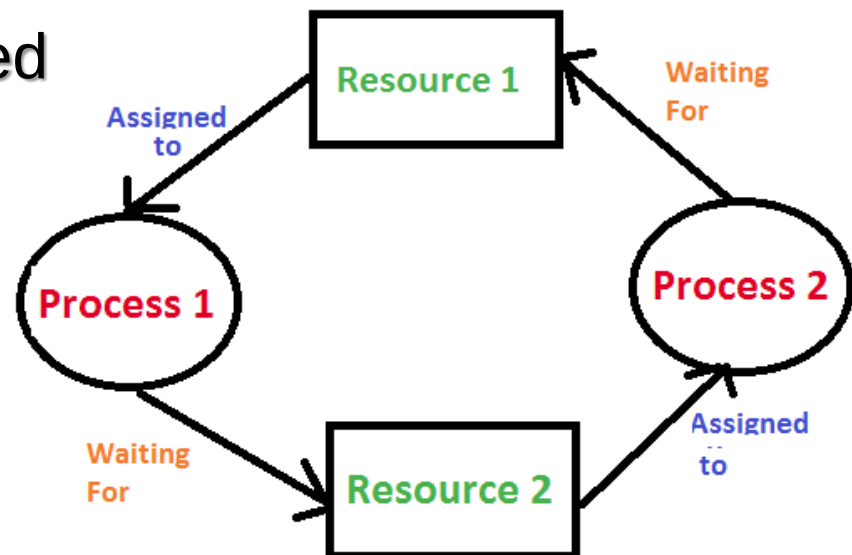


Resource Management in FreeRTOS with Mutexes

Deadlock:

3. Task B successfully takes mutex Y before attempting to also take mutex X but mutex X is held by Task A so is not available to Task B.

Task B enters the Blocked state to wait for mutex X to be released.

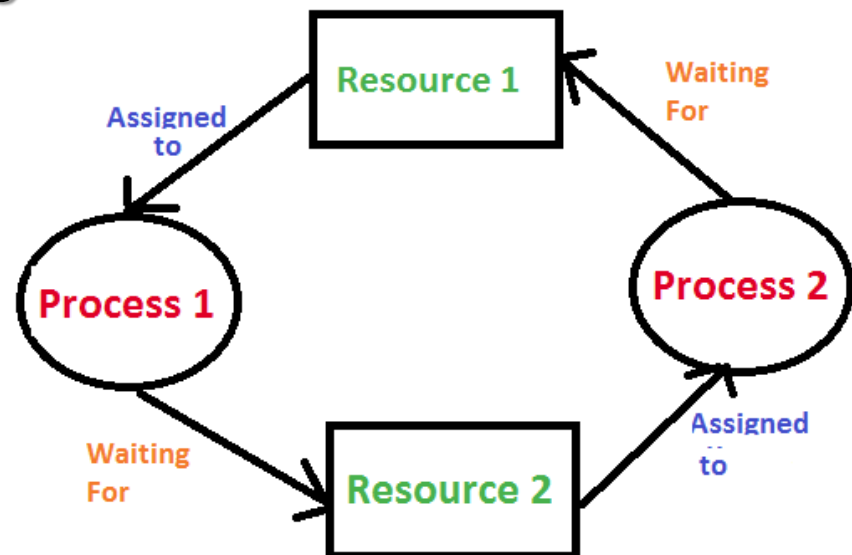




Resource Management in FreeRTOS with Mutexes

Deadlock:

- Task A continues executing and attempts to take mutex Y but mutex Y is held by Task B, so is not available to Task A. Task A enters the Blocked state to wait for mutex Y to be released.

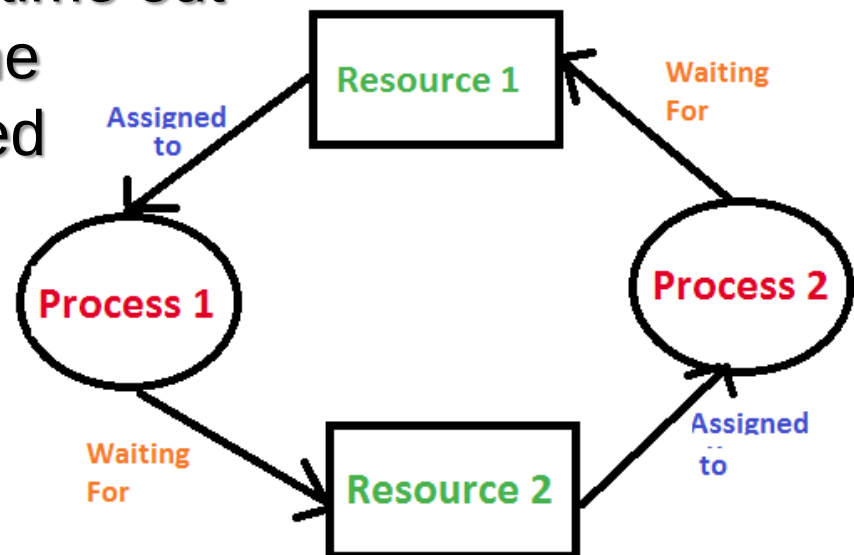




Resource Management in FreeRTOS with Mutexes

Task A is waiting for a mutex held by Task B, and Task B is waiting for a mutex held by Task A. Deadlock has occurred because neither task can proceed.

To avoid a deadlock use a time out that is a little longer than the maximum time it is expected to have to wait for the mutex.

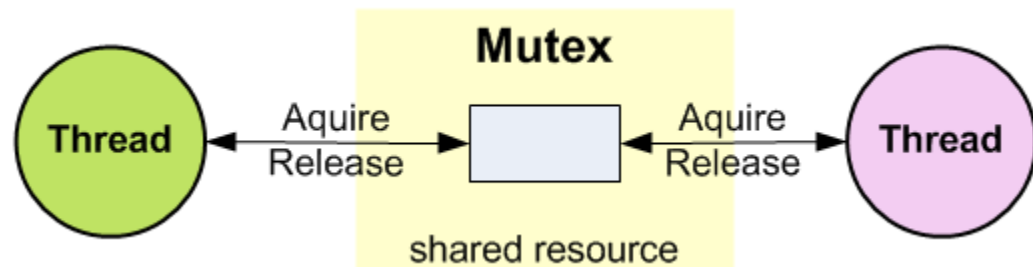


Resource Management in FreeRTOS with Mutexes



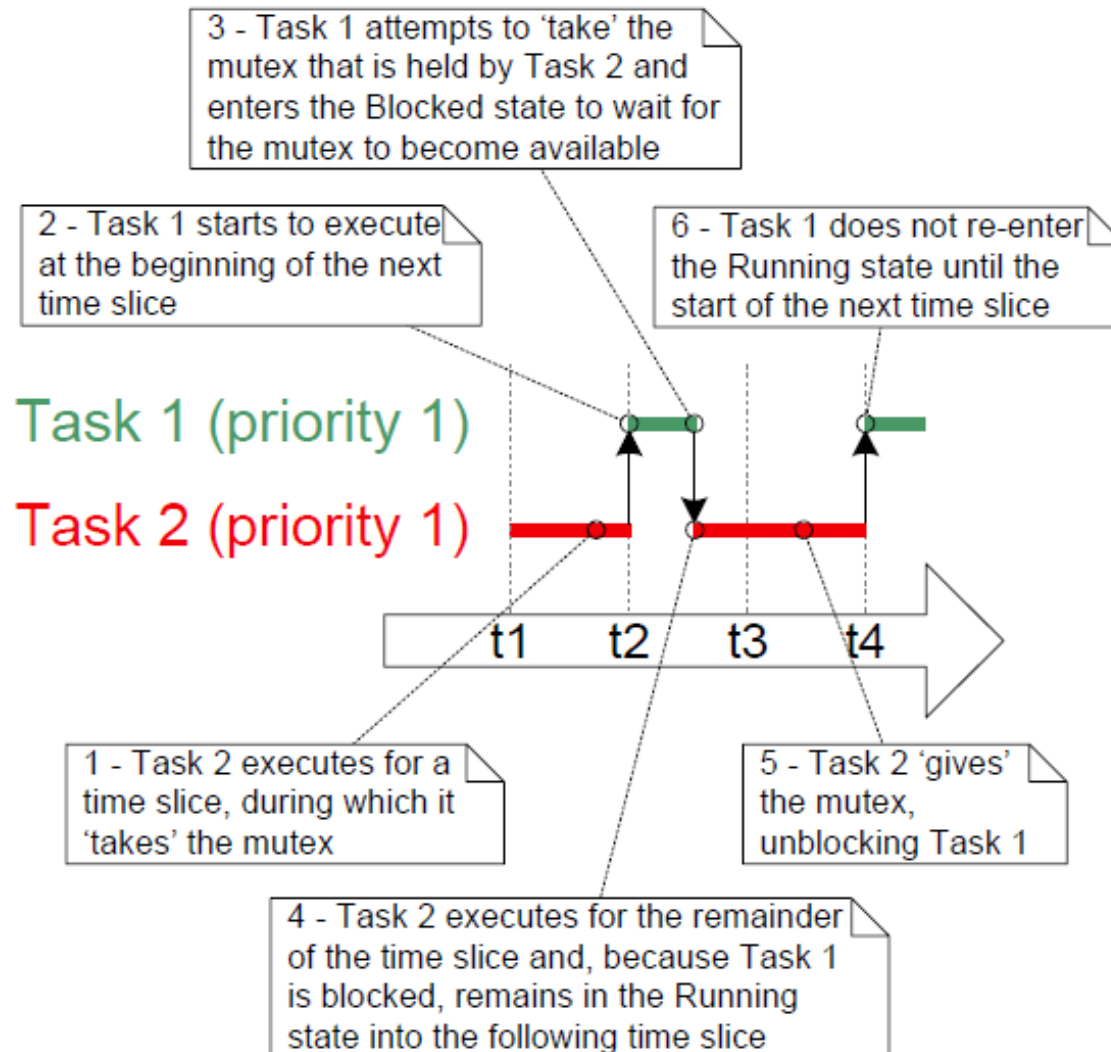
If Task 1 and Task 2 have the same priority and Task 1 is in the Blocked state to wait for a mutex that is held by Task 2, then Task 1 will not pre-empt Task 2 when Task 2 'gives' the mutex.

Instead, Task 2 will remain in the Running state, and Task 1 will simply move from the Blocked state to the Ready state.





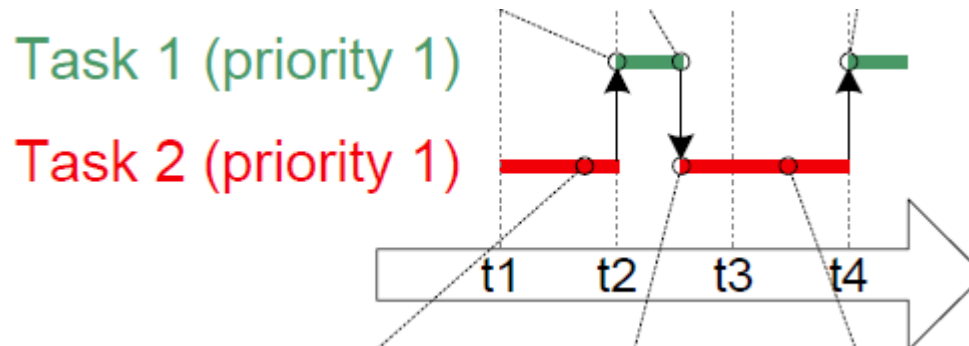
Resource Management in FreeRTOS with Mutexes





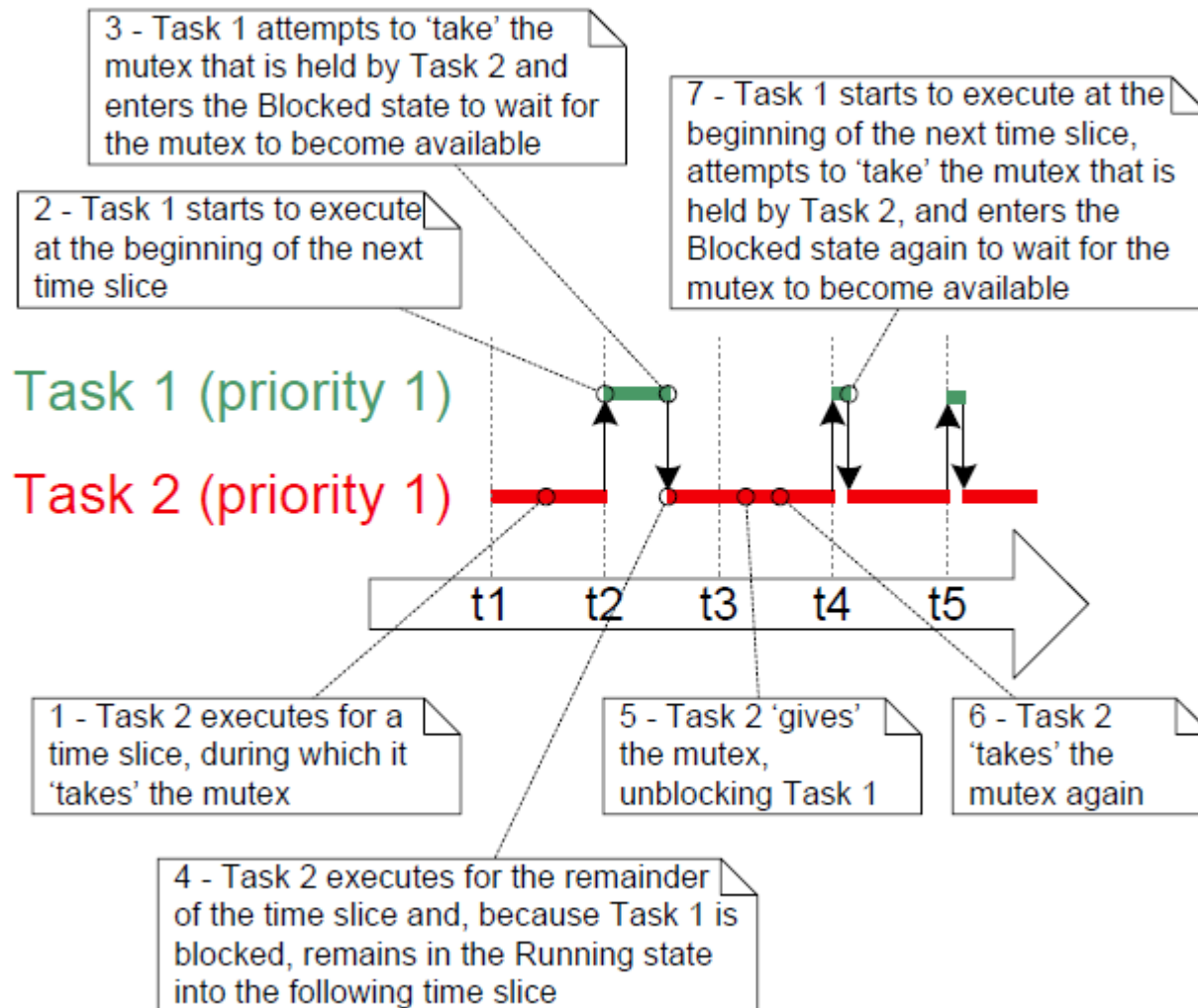
Resource Management in FreeRTOS with Mutexes

If a mutex is used in a tight loop by more than one task, and the tasks that use the mutex have the same priority, then care must be taken to ensure the tasks receive an approximately equal amount of processing time.





Resource Management in FreeRTOS with Mutexes

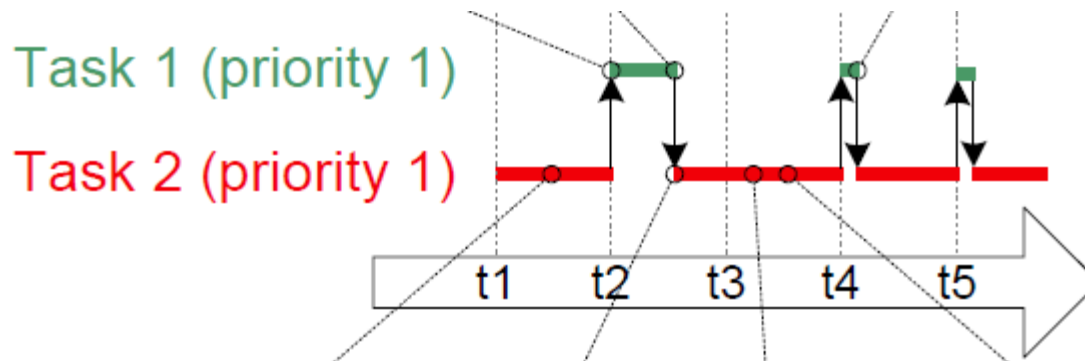




Resource Management in FreeRTOS with Mutexes

The Figure demonstrates that Task 1 will be prevented from obtaining the mutex until the start of a time slice coincides with one of the short periods during which Task 2 is not the mutex holder.

This scenario can be avoided by adding a call to *taskYIELD()* after the call to *xSemaphoreGive()*.





End of the Introduction to FreeRTOS

But so much more to learn using the FreeRTOS simulator on Windows that requires no external hardware.

Chapter 6 – Interrupt Management

Chapter 8 – Event Groups

Chapter 9 – Task Notifications

